



UNIVERSIDAD PONTIFICIA CATÓLICA DE VALPARAÍSO

Proyecto II

PROCESAMIENTO DIGITAL MULTIMEDIA - EIE401

Profesor: Jorge Cárdenas

1. Detección y medición de objetos astronómicos en imágenes ruidosas

Las imágenes astronómicas suelen contener objetos débiles que quedan parcialmente ocultos por ruido del detector, gradientes de fondo, píxeles calientes, saturación y desenfoque óptico. Antes de realizar una interpretación científica, la imagen debe corregirse, filtrarse, segmentarse y medirse. Este problema aparece en observatorios, telescopios robóticos, búsqueda de objetos débiles y generación de catálogos astronómicos [1, 3].

En este proyecto trabajarás con un conjunto de imágenes sintéticas de tipo telescópico, de tamaño 512×512 píxeles. Cada imagen contiene objetos brillantes y débiles sobre un fondo no uniforme. Junto con el conjunto de imágenes se entrega un catálogo verdadero con la posición y algunas propiedades de cada objeto. Tu objetivo no es clasificar imágenes ni entrenar modelos de aprendizaje automático. Tu objetivo es construir un flujo clásico de procesamiento digital de imágenes para detectar, contar y medir objetos astronómicos.

El procedimiento esperado está inspirado en herramientas astronómicas de extracción de fuentes como SExtractor, donde los objetos se detectan mediante umbrales sobre el fondo y posteriormente se miden sus propiedades [1]. Para la etapa de segmentación se recomienda estudiar métodos clásicos de umbralización, por ejemplo el criterio de Otsu [2], y herramientas reproducibles de procesamiento de imágenes en Python como scikit-image [4]. Como guía práctica, la documentación de Photutils describe un flujo de detección por segmentación que incluye estimación de fondo, umbral sobre el ruido, píxeles conectados, centroides y fotometría [3].



Figura 1: Ejemplo del problema. La imagen observada contiene ruido, fondo no uniforme y objetos débiles. El catálogo verdadero se entrega para evaluar el desempeño del algoritmo.

Matemáticamente, una imagen observada puede escribirse como

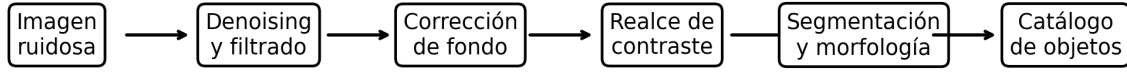
$$I_{\text{obs}}(x, y) = B(x, y) + \sum_{k=1}^{N_s} S_k(x, y) + n(x, y) + h(x, y), \quad (1)$$

donde $B(x, y)$ es el fondo, $S_k(x, y)$ representa el objeto astronómico k , $n(x, y)$ es ruido y $h(x, y)$ representa artefactos como píxeles calientes. Debes estimar una imagen corregida

$$I_{\text{corr}}(x, y) = I_{\text{obs}}(x, y) - \hat{B}(x, y), \quad (2)$$

y luego obtener una máscara binaria

$$M(x, y) = \begin{cases} 1, & \text{objeto detectado,} \\ 0, & \text{fondo.} \end{cases} \quad (3)$$



Salida evaluada: conteo de objetos y error de centroide

Figura 2: Flujo mínimo esperado para el proyecto. La solución debe basarse en procesamiento clásico de imágenes: filtrado, corrección de fondo, realce, segmentación, morfología y componentes conectados.

Tareas requeridas

Tu notebook debe incluir, como mínimo, las siguientes etapas:

1. Cargar el conjunto de imágenes y el archivo `true_catalog.csv`. Visualizar ejemplos, histogramas y perfiles de intensidad.
2. Aplicar filtrado y denoising. Debes comparar al menos tres métodos, por ejemplo filtro promedio, filtro gaussiano, filtro de mediana o filtro bilateral.
3. Estimar y remover el fondo usando una técnica de tu elección: filtro gaussiano ancho, filtro de mediana ancho, apertura morfológica o ajuste suave.
4. Aplicar realce de contraste: normalización, ecualización de histograma, ecualización adaptativa, transformación gamma o transformación logarítmica.
5. Segmentar los objetos mediante umbral global, umbral adaptativo, método de Otsu o un umbral basado en la media y la desviación estándar del fondo.
6. Limpiar la máscara mediante operaciones morfológicas: erosión, dilatación, apertura, cierre, relleno de huecos o eliminación de regiones pequeñas.
7. Etiquetar componentes conectados y medir cada objeto detectado: centroide, área, brillo total y brillo medio.
8. Calcular las métricas obligatorias y reportarlas claramente en el notebook y en el informe.

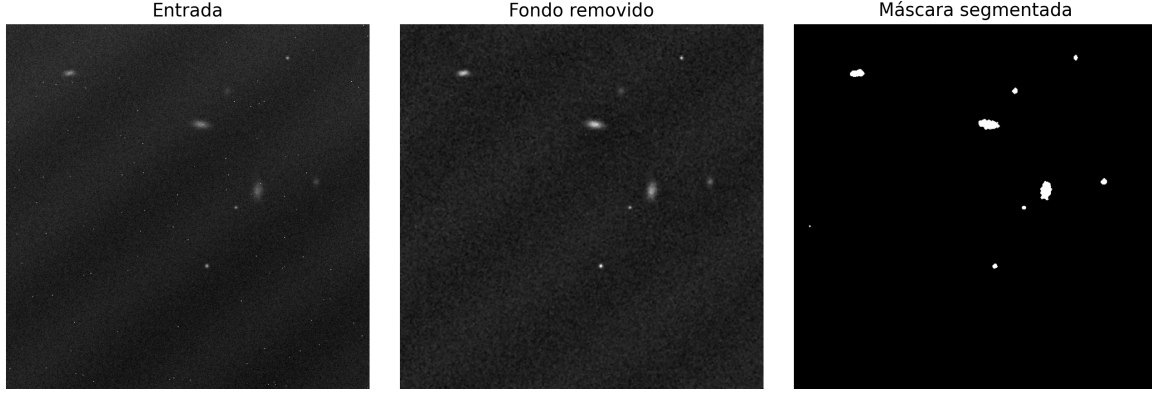


Figura 3: Ejemplo conceptual de corrección de fondo y segmentación. La solución final de cada estudiante puede ser diferente, pero debe justificar cada etapa del procesamiento.

Métricas obligatorias

Debes reportar, por lo menos, las siguientes dos métricas. Para cada imagen, el error en el número de objetos detectados es

$$E_N = |N_{\text{pred}} - N_{\text{true}}|. \quad (4)$$

Para cada objeto correctamente emparejado con el catálogo verdadero, el error de centroide es

$$e_c^{(i)} = \sqrt{(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2}. \quad (5)$$

Debes reportar el error medio o mediano de centroide en píxeles. Puedes considerar que una detección es correcta si el centroide predicho queda dentro de una tolerancia r_{tol} respecto a un objeto verdadero. Debes explicar el valor de r_{tol} usado.

Importante: no se permite usar redes neuronales, modelos preentrenados, clasificadores de aprendizaje automático ni bibliotecas como PyTorch o TensorFlow. Este proyecto debe resolverse con procesamiento digital de imágenes.

Evidencia visual, reporte y presentación

No es necesario entregar una carpeta aparte con imágenes procesadas ni un catálogo final en formato `.csv`. La evaluación se realizará a partir del notebook, el repositorio, el informe y la presentación oral. Sin embargo, debes incluir en el notebook y en el reporte hasta 10 ejemplos representativos: detecciones correctas, falsos positivos y falsos negativos. Las métricas del notebook deben coincidir con las métricas reportadas en el informe.

Además, cada estudiante o grupo deberá preparar una **presentación oral de 15 minutos** sobre el procedimiento desarrollado y los resultados obtenidos. La presentación debe explicar, de forma clara y ordenada, cómo se resolvió el problema, qué decisiones de procesamiento se tomaron y qué tan bien funcionó el método propuesto.

La presentación debe incluir, como mínimo:

1. descripción breve del problema y del conjunto de imágenes;
2. diagrama o resumen del flujo de procesamiento implementado;
3. explicación de las etapas principales: filtrado, estimación de fondo, realce, segmentación, morfología y componentes conectados;
4. resultados cuantitativos usando, al menos, E_N y el error de centroide;
5. ejemplos visuales de detecciones correctas, falsos positivos y falsos negativos;

6. discusión breve sobre las limitaciones del método y posibles mejoras.

Se recomienda organizar la presentación en aproximadamente: 2 minutos para el problema y los datos, 4 minutos para la metodología, 5 minutos para resultados y ejemplos visuales, 2 minutos para discusión de errores, y 2 minutos para conclusiones.

■ **Desarrollo de la solución (60 %):**

- El trabajo debe realizarse en Jupyter Notebook. Se evaluará el orden, la estructura del código, los comentarios y las explicaciones del procedimiento.
- Debes incluir los archivos necesarios para ejecutar tu solución, incluyendo el script de generación o descarga del conjunto de imágenes.
- La solución debe reposar en tu repositorio personal en GitHub.
- La hora máxima de entrega será las 23:59 de la fecha establecida. Cualquier retraso se sancionará con un 10 % de la nota final del proyecto.

■ **Reporte (25 %):**

- Debe realizarse en formato de paper IEEE. Se valorará la calidad del texto, la explicación del procedimiento y la discusión de resultados.
- El informe NO puede superar 2 páginas de contenido, sin incluir referencias.
- Debe contener abstract, introducción, marco teórico, metodología, resultados y conclusiones.
- Debe incluir por lo menos cuatro referencias.
- El reporte debe nombrarse `Apellido_Nombre_cedula_Proyecto2.pdf`.

■ **Presentación oral (15 %):**

- La presentación debe durar 15 minutos.
- Debe mostrar el flujo de procesamiento, los resultados cuantitativos y ejemplos visuales representativos.
- Se evaluará la claridad de la explicación, la justificación técnica de las decisiones tomadas, la interpretación de errores y el cumplimiento del tiempo asignado.

Referencias

- [1] Emmanuel Bertin y Stéphane Arnouts. “SExtractor: Software for source extraction”. En: *Astronomy and Astrophysics Supplement Series* 117 (1996), págs. 393-404. DOI: 10.1051/aas:1996164.
- [2] Nobuyuki Otsu. “A Threshold Selection Method from Gray-Level Histograms”. En: *IEEE Transactions on Systems, Man, and Cybernetics* 9.1 (1979), págs. 62-66. DOI: 10.1109/TSMC.1979.4310076.
- [3] Photutils Development Team. *Image Segmentation and Source Detection*. Consultado el 7 de mayo de 2026. 2026. URL: https://photutils.readthedocs.io/en/stable/user_guide/segmentation.html.
- [4] Stefan van der Walt et al. “scikit-image: image processing in Python”. En: *PeerJ* 2 (2014), e453. DOI: 10.7717/peerj.453.